计算机学院实验报告

实验题目： 插值与 Blinn-Phong 着色模型		学号：202200130077
日期：2024.5.7	班级：智能	姓名：薛润泽
Email：		

实验目的：

1. 修改函数 rasterize_triangle(const Triangle& t) in rasterizer.cpp: 在此 处实现与作业 2 类似的插值算法，实现法向量、颜色、纹理颜色的插值。
2. 修改函数 get_projection_matrix() in main.cpp: 将你自己在之前的实验中 实现的投影矩阵填到此处，此时你可以运行 ./Rasterizer output.png normal 来观察法向量实现结果。
3. 修改函数 phong_fragment_shader() in main.cpp: 实现 Blinn-Phong 模型计算 Fragment Color.
4. 修改函数 texture_fragment_shader() in main.cpp: 在实现 Blinn-Phong 的基础上，将纹理颜色视为公式中的 kd，实现 Texture Shading Fragment Shader.
5. 修改函数 bump_fragment_shader() in main.cpp: 在实现 Blinn-Phong 的 基础上，仔细阅读该函数中的注释，实现 Bump mapping.
6. 修改函数 displacement_fragment_shader() in main.cpp: 在实现 Bump mapping 的基础上，实现 displacement mapping.

实验环境介绍：

Ubuntu

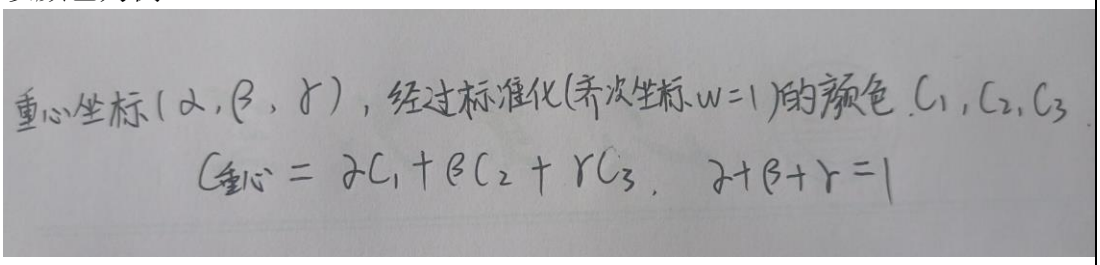
解决问题的主要思路：

本实验的主要思路如下：

1. 学习重心坐标插值法，对三角形内的像素颜色，深度，法线进行插值计算。
2. 学习 Blinn-Phong 模型，使用环境光、漫反射光、镜面反射光进行着色。
3. 学习 Bump-Mapping 和 Displacement-Mapping，了解使用凹凸贴图对模型产生视觉上的凹凸起伏。
4. 使用双线性插值完成 Texture 寻点

实验步骤与实验结果：

1、重心坐标插值法：
我们已知三角形三个顶点的颜色、法线等的信息，但是三角形内可能会有多个像素点。怎样使得三角形内的特征变化平滑？通过使用插值的方法，利用三角形内像素的位置，通过计算重心坐标，得到颜色、法线等的插值信息。
以颜色为例：



2、Blinn-Phong 模型
Bilinn-Phong 模型是对物体反射光的建模。模型主要分为 3 部分，分别理想漫反射（diffuse）、镜面反射光（specular）、环境光（ambient）
其中，理想漫反射公式如下图

$$I_d = I_p K_d \cos\theta$$
$$\cos\theta = L\ N$$
$$I_d = I_p K_d L \cdot N$$

L 为入射光（方向为从入射点到光源）的单位向量，N 为法向单位向量
所以

$$I_d = I_p K_d L \cdot N$$

其中，镜面反射光公式如下图

$$I_s = I_p\ K_s \cos\alpha^n$$
$$\cos\alpha = H \cdot N$$
$$I_s = I_p\ K_s H \cdot N^n$$

H 为半程向量方向向量，N 为法向单位向量
其中，环境光公式如下

$$I_e = I_a K_a$$

则，对于一个光源照射物体，视角中得到的光照明模型公式为

$$I = I_p K_d L \cdot N + I_p\ K_s H \cdot N^n I_a K_a$$

对于多个点产生的光照明模型公式为 I 的加和。

3、实验中 testure_fragment_shader 部分，对于贴图像素的定位，Texture 类已经实现了函数 getColor 来实现贴图像素的选择。对于贴图部分，我们也需要光照模型的参与，实现光线对模型作用后，在视线中的效果展现。
4、对于 Bump_fragment_shader 部分，Bump 凹凸贴图是一张灰度图，记录的是每个像素点的高度，凹凸贴图的原理是通过高度信息“扰动”射过来的光线，然后使得 phong 模型中的镜面光、漫反射光发生扰动，实现高低不平的视觉效果。在代码中，实现了 TBN 矩阵，TBN 矩阵是一个将切线空间变化到物体空

间的变换矩阵，TBN 矩阵与颜色变化率向量做乘积就可以得到“受到扰动”后物体空间中的法线向量。

5、Displacement Mapping 在 Bump Mapping 的基础上，实际改变了物体表面的凹凸状况，将点沿着法线的方向进行了移动，并且添加了实际光照。

6、双线性插值找到映射到纹理处点临近的四个纹理点，按照位置信息，完成三次两段双线性插值，返回插值结果，使得模型贴膜颜色更加平滑。

// 投影矩阵 如下

```
Eigen::Matrix4f get_projection_matrix(float eye_fov, float
aspect_ratio, float zNear, float zFar)
{
    // TODO: Use the same projection matrix from the previous
assignments
    float n = zNear;
    float f = zFar;
    float t = zNear*tan(eye_fov/2);
    float b = -zNear*tan(eye_fov/2);
    float r = aspect_ratio*zNear*tan(eye_fov/2);
    float l = -aspect_ratio*zNear*tan(eye_fov/2);
    Eigen::Matrix4f projection = Eigen::Matrix4f::Identity();

    projection<<
    2*n/(r-l), 0, (r + l)/(r - l), 0,
    0, 2*n/(t-b), (t+b)/(t-b), 0,
    0, 0, -(f + n)/(f - n), -2*f*n/(f - n),
    0, 0, -1, 0;

    return projection;
}
```

// 插值函数如下

```
void rst::rasterizer::rasterize_triangle(const Triangle& t,
const std::array<Eigen::Vector3f, 3>& view_pos)
{
    auto v = t.toVector4();
    int leftx = 0x3f3f3f3f, lefty = 0x3f3f3f3f, rightx = -
0x3f3f3f3f, righty = -0x3f3f3f3f;// the rectangle cover
triangle
    for(auto it  : v){
        if(leftx > it[0]) leftx = it[0];
        if(lefty > it[1]) lefty = it[1];
        if(rightx < it[0]) rightx = it[0];
        if(righty < it[1]) righty = it[1];
        for(int y = lefty; y <= righty; y++){
```

```
          for(int x = leftx; x <= rightx; x++){
              double  kx = (double) x + 0.5; double ky =
(double) y + 0.5;
              if(insideTriangle(kx, ky, t.v)){// check the
pixel in or not in the triangle
                  auto weightpoint = computeBarycentric2D(kx,
ky, t.v);
                  double alpha =  std::get<0>(weightpoint);
                  double beta = std::get<1>(weightpoint);
                  double gamma = std::get<2>(weightpoint);
                  double z_interpolated = alpha * v[0].z() /
v[0].w() + beta * v[1].z()/v[1].w() + gamma *
v[2].z()/v[2].w();
                  double normalization_coe = 1 / (alpha /
v[0].w() + beta / v[1].w() + gamma / v[2].w());
                  z_interpolated *= normalization_coe; //
normalization depth
                  if (z_interpolated < depth_buf[get_index(x,
y)]){ // can indeed be push on screen
                      Eigen::Vector2i p = { (float)x,(float)y};
                      auto interpolated_color =
interpolate(alpha, beta, gamma, t.color[0], t.color[1],
t.color[2], 1);
                      auto interpolated_normal =
interpolate(alpha, beta, gamma, t.normal[0], t.normal[1],
t.normal[2], 1);
                      auto interpolated_texcoords =
interpolate(alpha, beta, gamma, t.tex_coords[0],
t.tex_coords[1], t.tex_coords[2], 1);
                      auto interpolated_shadingcoords =
interpolate(alpha, beta, gamma, view_pos[0], view_pos[1],
view_pos[2], 1);
                      fragment_shader_payload
payload(interpolated_color, interpolated_normal.normalized(),
interpolated_texcoords, texture ? &*texture : nullptr);
                      payload.view_pos =
interpolated_shadingcoords;
                      auto pixel_color =
fragment_shader(payload);
                      set_pixel(p, pixel_color);
                      depth_buf[get_index(x, y)] =
z_interpolated;
                  }
              }
```

```
            }
        }
    }


}
```

// Blinn – phong 光线模型

```
Eigen::Vector3f phong_fragment_shader(const
fragment_shader_payload& payload)
{
    Eigen::Vector3f ka = Eigen::Vector3f(0.005, 0.005, 0.005);
    Eigen::Vector3f kd = payload.color;
    Eigen::Vector3f ks = Eigen::Vector3f(0.7937, 0.7937,
0.7937);

    auto l1 = light{{20, 20, 20}, {500, 500, 500}};
    auto l2 = light{{-20, 20, 0}, {500, 500, 500}};

    std::vector<light> lights = {l1, l2};
    Eigen::Vector3f amb_light_intensity{10, 10, 10};
    Eigen::Vector3f eye_pos{0, 0, 10};

    float p = 150;

    Eigen::Vector3f color = payload.color;
    Eigen::Vector3f point = payload.view_pos;
    Eigen::Vector3f normal = payload.normal;

    Eigen::Vector3f result_color = {0, 0, 0};
    for (auto& light : lights)
    {
        // TODO: For each light source in the code, calculate
what the *ambient*, *diffuse*, and *specular*
        // components are. Then, accumulate that result on the
*result_color* object.
        auto inlight = light.position - point; // inlight
        auto r_squre = inlight.dot(inlight);
        auto ambient = ka.cwiseProduct(amb_light_intensity);
        auto diffuse = kd.cwiseProduct(light.intensity /
r_squre) * std::max(0.0f ,
inlight.normalized().dot(normal.normalized()));
        auto outview = eye_pos - point;
        auto half_vector = (inlight + outview).normalized();
```

```
        auto specular = ks.cwiseProduct(light.intensity /
r_squre) *
std::pow(std::max(0.0f ,(half_vector.normalized()).dot(normal.
normalized())), p);
        result_color += ambient + diffuse + specular;
    }
    return result_color * 255.f;
}
```

// texture 实现

```
Eigen::Vector3f texture_fragment_shader(const
fragment_shader_payload& payload)
{
    Eigen::Vector3f return_color = {0, 0, 0};
    if (payload.texture)
    {
        // TODO: Get the texture value at the texture
coordinates of the current fragment
        return_color =
payload.texture->getColor(payload.tex_coords.x(),
payload.tex_coords.y());

    }
    Eigen::Vector3f texture_color;
    texture_color << return_color.x(), return_color.y(),
return_color.z();

    Eigen::Vector3f ka = Eigen::Vector3f(0.005, 0.005, 0.005);
    Eigen::Vector3f kd = texture_color / 255.f;
    Eigen::Vector3f ks = Eigen::Vector3f(0.7937, 0.7937,
0.7937);

    auto l1 = light{{20, 20, 20}, {500, 500, 500}};
    auto l2 = light{{-20, 20, 0}, {500, 500, 500}};

    std::vector<light> lights = {l1, l2};
    Eigen::Vector3f amb_light_intensity{10, 10, 10};
    Eigen::Vector3f eye_pos{0, 0, 10};

    float p = 150;

    Eigen::Vector3f color = texture_color;
    Eigen::Vector3f point = payload.view_pos;
    Eigen::Vector3f normal = payload.normal;
```

```
    Eigen::Vector3f result_color = {0, 0, 0};

    for (auto& light : lights)
    {
        // TODO: For each light source in the code, calculate
what the *ambient*, *diffuse*, and *specular*
        // components are. Then, accumulate that result on the
*result_color* object.
        auto inlight = light.position - point; // inlight
        auto r_squre = inlight.dot(inlight);
        auto ambient = ka.cwiseProduct(amb_light_intensity);
        auto diffuse = kd.cwiseProduct(light.intensity /
r_squre) * std::max(0.0f ,
inlight.normalized().dot(normal.normalized()));
        auto outview = eye_pos - point;
        auto half_vector = (inlight + outview).normalized();
        auto specular = ks.cwiseProduct(light.intensity /
r_squre) *
std::pow(std::max(0.0f ,(half_vector.normalized()).dot(normal.
normalized())), p);
        result_color += ambient + diffuse + specular;
    }

    return result_color * 255.f;
}
```

// Bump Mapping 实现

```
Eigen::Vector3f bump_fragment_shader(const
fragment_shader_payload& payload)
{

    Eigen::Vector3f ka = Eigen::Vector3f(0.005, 0.005, 0.005);
    Eigen::Vector3f kd = payload.color;
    Eigen::Vector3f ks = Eigen::Vector3f(0.7937, 0.7937,
0.7937);

    auto l1 = light{{20, 20, 20}, {500, 500, 500}};
    auto l2 = light{{-20, 20, 0}, {500, 500, 500}};

    std::vector<light> lights = {l1, l2};
    Eigen::Vector3f amb_light_intensity{10, 10, 10};
```

```
    Eigen::Vector3f eye_pos{0, 0, 10};

    float p = 150;

    Eigen::Vector3f color = payload.color;
    Eigen::Vector3f point = payload.view_pos;
    Eigen::Vector3f normal = payload.normal;


    float kh = 0.2, kn = 0.1;

    // TODO: Implement bump mapping here
    // Let n = normal = (x, y, z)
    // Vector t =
(x*y/sqrt(x*x+z*z),sqrt(x*x+z*z),z*y/sqrt(x*x+z*z))
    // Vector b = n cross product t
    // Matrix TBN = [t b n]
    // dU = kh * kn * (h(u+1/w,v)-h(u,v))
    // dV = kh * kn * (h(u,v+1/h)-h(u,v))
    // Vector ln = (-dU, -dV, 1)
    // Normal n = normalize(TBN * ln)

    auto normal_x = normal.x();
    auto normal_y = normal.y();
    auto normal_z = normal.z();
    auto sqrt_xs_zs = std::sqrt(normal_x * normal_x + normal_z
* normal_z);
    // if(sqrt_xs_zs == 0) return 0;
    Eigen::Vector3f tangent = {normal_x * normal_y /
sqrt_xs_zs, sqrt_xs_zs, normal_z*normal_y / sqrt_xs_zs };
    auto bi_tangent = normal.cross(tangent);
    Eigen::Matrix3f TBN;
    TBN <<
    tangent.x(), bi_tangent.x(), normal.x(),
    tangent.y(), bi_tangent.y(), normal.y(),
    tangent.z(), bi_tangent.z(), normal.z();
    auto u = payload.tex_coords.x();
    auto v = payload.tex_coords.y();
    auto w = payload.texture->width;
    auto h = payload.texture->height;
    auto dU = kh * kn *(payload.texture->getColor(u + 1.0f/w,
v).norm() - payload.texture->getColor(u, v).norm());
    auto dV = kh * kn *(payload.texture->getColor(u, v +
1.0f/h).norm() - payload.texture->getColor(u, v).norm());
```

```
    Eigen::Vector3f ln = {-dU, -dV, 1.0f};
    normal = (TBN * ln).normalized();
    Eigen::Vector3f result_color = {0, 0, 0};
    result_color = normal.normalized();

    return result_color * 255.f;
}
```

// Displacement Mapping 实现

```
Eigen::Vector3f displacement_fragment_shader(const
fragment_shader_payload& payload)
{

    Eigen::Vector3f ka = Eigen::Vector3f(0.005, 0.005, 0.005);
    Eigen::Vector3f kd = payload.color;
    Eigen::Vector3f ks = Eigen::Vector3f(0.7937, 0.7937,
0.7937);

    auto l1 = light{{20, 20, 20}, {500, 500, 500}};
    auto l2 = light{{-20, 20, 0}, {500, 500, 500}};

    std::vector<light> lights = {l1, l2};
    Eigen::Vector3f amb_light_intensity{10, 10, 10};
    Eigen::Vector3f eye_pos{0, 0, 10};

    float p = 150;

    Eigen::Vector3f color = payload.color;
    Eigen::Vector3f point = payload.view_pos;
    Eigen::Vector3f normal = payload.normal;

    float kh = 0.2, kn = 0.1;

    // TODO: Implement displacement mapping here
    // Let n = normal = (x, y, z)
    // Vector t =
(x*y/sqrt(x*x+z*z),sqrt(x*x+z*z),z*y/sqrt(x*x+z*z))
    // Vector b = n cross product t
    // Matrix TBN = [t b n]
    // dU = kh * kn * (h(u+1/w,v)-h(u,v))
    // dV = kh * kn * (h(u,v+1/h)-h(u,v))
    // Vector ln = (-dU, -dV, 1)
    // Position p = p + kn * n * h(u,v)
```

```
    // Normal n = normalize(TBN * ln)

    auto normal_x = normal.x();
    auto normal_y = normal.y();
    auto normal_z = normal.z();
    auto sqrt_xs_zs = std::sqrt(normal_x * normal_x + normal_z
* normal_z);
    // if(sqrt_xs_zs == 0) return 0;
    Eigen::Vector3f tangent = {normal_x * normal_y /
sqrt_xs_zs, sqrt_xs_zs, normal_z*normal_y / sqrt_xs_zs };
    auto bi_tangent = normal.cross(tangent);
    Eigen::Matrix3f TBN;
    TBN <<
    tangent.x(), bi_tangent.x(), normal.x(),
    tangent.y(), bi_tangent.y(), normal.y(),
    tangent.z(), bi_tangent.z(), normal.z();
    auto u = payload.tex_coords.x();
    auto v = payload.tex_coords.y();
    auto w = payload.texture->width;
    auto h = payload.texture->height;
    auto dU = kh * kn *(payload.texture->getColor(u + 1.0f/w,
v).norm() - payload.texture->getColor(u, v).norm());
    auto dV = kh * kn *(payload.texture->getColor(u, v +
1.0f/h).norm() - payload.texture->getColor(u, v).norm());
    Eigen::Vector3f ln = {-dU, -dV, 1.0f};
    point += (kn * normal * payload.texture->getColor(u,
v).norm());
    normal = (TBN * ln).normalized();
    Eigen::Vector3f result_color = {0, 0, 0};

    for (auto& light : lights)
    {
        // TODO: For each light source in the code, calculate
what the *ambient*, *diffuse*, and *specular*
        // components are. Then, accumulate that result on the
*result_color* object.
        auto inlight = light.position - point; // inlight
        auto r_squre = inlight.dot(inlight);
        auto ambient = ka.cwiseProduct(amb_light_intensity);
        auto diffuse = kd.cwiseProduct(light.intensity /
r_squre) * std::max(0.0f ,
inlight.normalized().dot(normal.normalized()));
        auto outview = eye_pos - point;
        auto half_vector = (inlight + outview).normalized();
```

```
        auto specular = ks.cwiseProduct(light.intensity /
r_squre) *
std::pow(std::max(0.0f ,(half_vector.normalized()).dot(normal.
normalized())), p);
        result_color += ambient + diffuse + specular;

    }

    return result_color * 255.f;
}
```

// Texture 类中 双线性插值实现

```
    Eigen::Vector3f getColorBilinear(float u, float v){
        u = std:: max(0.0f, u); u = std::min(1.0f, u);
        v = std:: max(0.0f, v); v = std::min(1.0f, v);

        auto u_img = u * width;
        auto v_img = (1 - v) * height;
        auto u_img1 = (int)(u_img);
        auto u_img2 = u_img1 + 1;
        auto v_img1 = (int)(v_img);
        auto v_img2 = v_img1 + 1;
        float k1 = v_img - v_img1;
        float k2 = u_img - u_img1;
        auto color1 = image_data.at<cv::Vec3b>(v_img1, u_img1);
        auto color2 = image_data.at<cv::Vec3b>(v_img2, u_img1);

        auto color3 = image_data.at<cv::Vec3b>(v_img1, u_img2);
        auto color4 = image_data.at<cv::Vec3b>(v_img2, u_img2);

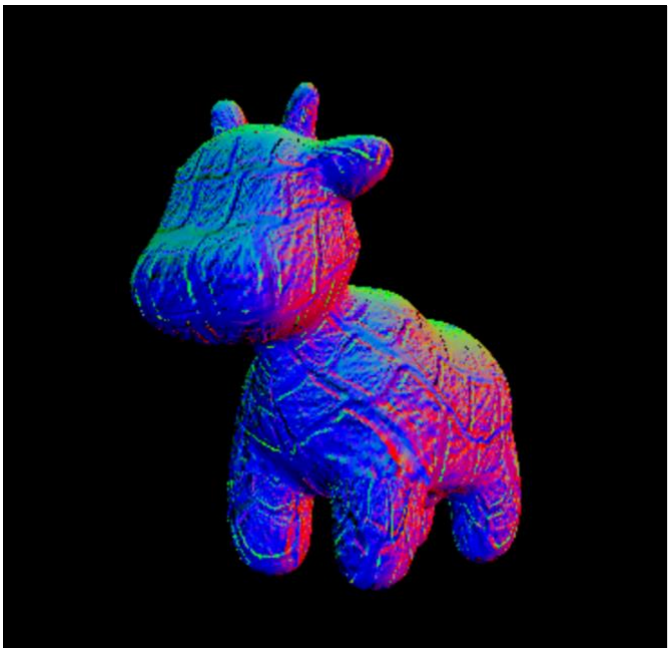
        auto color5 = k1 * color1 + (1.0f - k1) * color2;
        auto color6 = k1 * color3 + (1.0f - k1) * color4;
        auto color = k2 * color5 + (1.0f - k2) * color6;
        return Eigen::Vector3f(color[0], color[1], color[2]);
    }
```

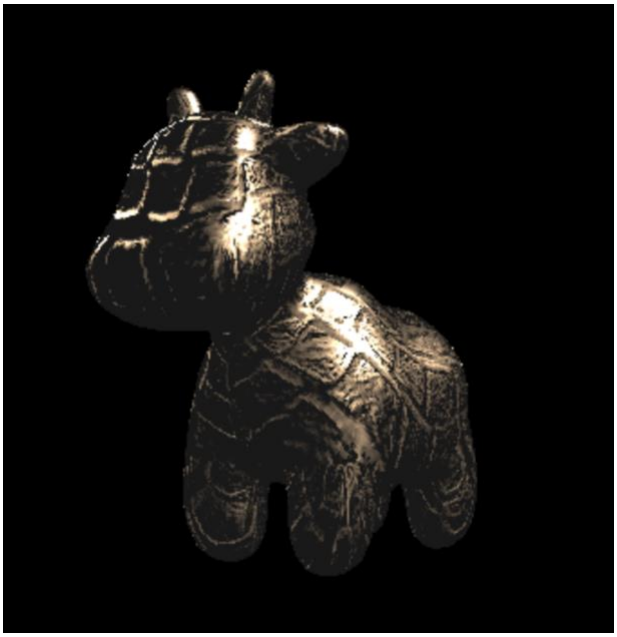
实验效果：



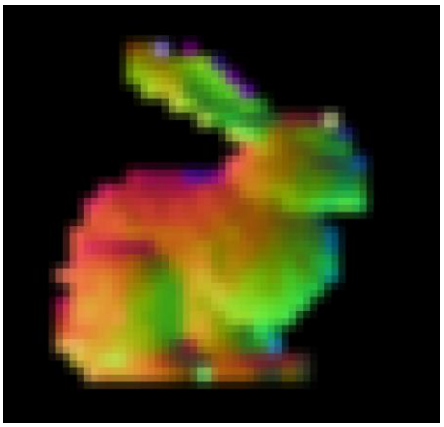






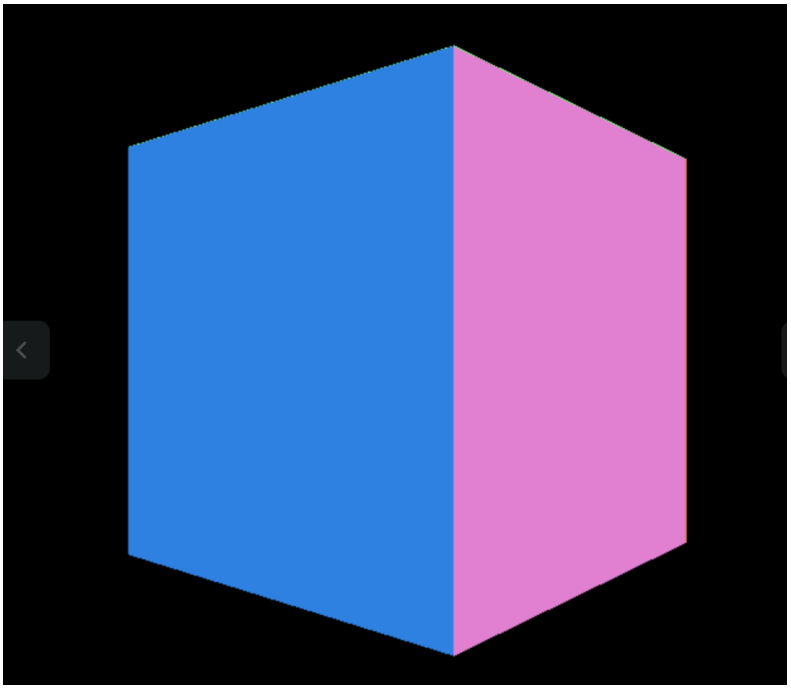


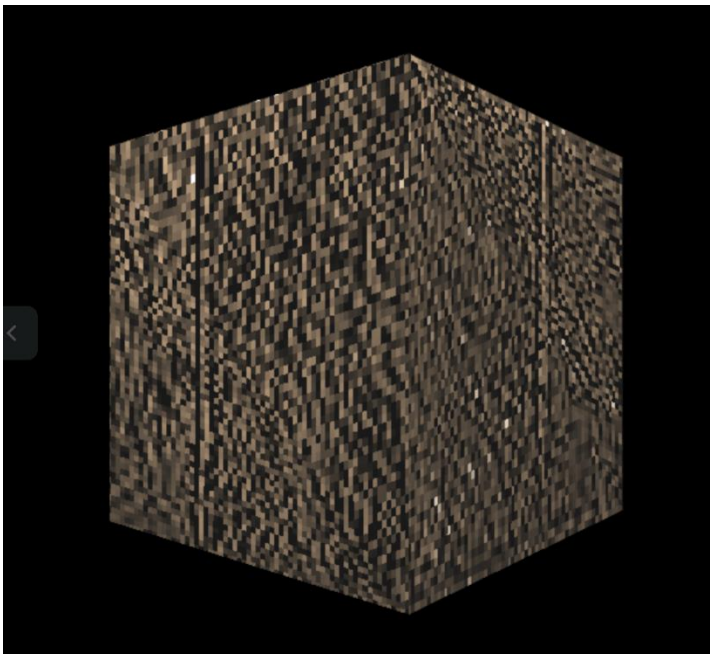
更换为 bunny 的模型



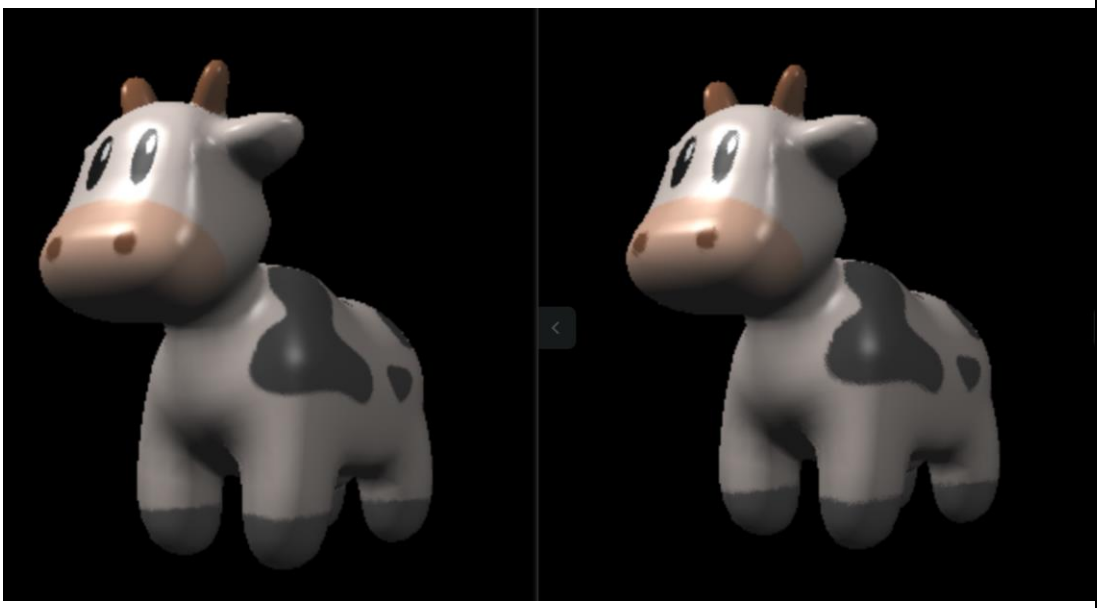
更换为 Cube 的模型，调用 texture，normal，phong 等命令







普通 texture 寻点与双线性插值寻点对比（左侧普通，右侧双线性插值）



实验中存在的问题及解决：

1、我在实验中遇到了 opencv 的通道报错

因为没有选择新建 build 文件夹，而是把 make 的文件直接放到了 code 文件夹下。代码中的模型寻址为 ../models 自动寻找父类文件夹下的目录，所以必须要把 make 出的可执行文件放到一个新建文件夹下。

2、更换模型时，生成图片后模型在图片中太小怎么办

可以选择放大模型或者降低视角

3、该纹理使用双线性插值进行 texture 寻点改善情况不大。